

# **Relatório – Resolução do Problema dos Missionários e Canibais por meio de Algoritmos de Busca sem Informação**

**Disciplina:** Inteligência Artificial

**Dupla:** Isaque Francisco e Francisco Yuri

## **1. INTRODUÇÃO**

Este relatório apresenta a modelagem e a resolução computacional do Problema dos Missionários e Canibais utilizando três estratégias de busca sem informação (ou busca cega): Busca em Largura (BFS), Busca em Profundidade (DFS) e Busca com Aprofundamento Iterativo (IDS).

O cenário avaliado consiste em transportar três missionários e três canibais localizados na margem inicial de um rio para a margem oposta. Para isso, há um barco com capacidade máxima para até duas pessoas por viagem. O desafio central reside em realizar essa transição respeitando rigidamente a restrição de segurança: em nenhuma das margens o número de canibais pode superar o de missionários (caso haja missionários presentes no local), garantindo que todos cheguem vivos e em segurança ao destino.

## 2. FORMALIZAÇÃO DO PROBLEMA:

O problema foi formulado de maneira precisa, estabelecendo apenas as distinções necessárias para assegurar uma solução computacional válida:

- **Representação do Estado:** O estado do sistema em qualquer instante é modelado por uma tupla ternária:

**(M, C, B)**

Onde:

**M:** Quantidade de missionários na margem inicial (esquerda), variando de 0 a 3.

**C:** Quantidade de canibais na margem inicial (esquerda), variando de 0 a 3.

**B:** Posição do barco, sendo 1 quando o barco está na margem inicial e 0 quando está na margem oposta.

- **Estado Inicial:**

**(3, 3, 1)**

Todos os três missionários, três canibais e o barco encontram-se na margem de origem.

- **Estado Objetivo:**

**(0, 0, 0)**

Todos os indivíduos e o barco transferidos com sucesso para a margem oposta.

- **Ações Possíveis (Operadores de Transição):**

Em cada travessia, o barco transporta no máximo duas pessoas e não pode navegar vazio. As variações permitidas ( $\Delta M$ ,  $\Delta C$ ) são:

- Transportar um missionário: (1, 0)
  - Transportar dois missionários: (2, 0)
  - Transportar um canibal: (0, 1)
  - Transportar dois canibais: (0, 2)
  - Transportar um missionário e um canibal: (1, 1)
- 
- **Restrições de Validez (Função de Validação):** Para que um estado gerado seja considerado seguro e válido, ele deve cumprir as seguintes regras simultâneas:
    1. 0 menor igual a M menor igual 3 e 0 menor igual C menor igual 3.
    2. Na margem inicial: Se  $M > 0$ , então  $M > \text{igual a } C$ .
    3. Na margem oposta: Se  $(3 - M) > 0$ , então  $(3 - M)$  maior igual a  $(3 - C)$ .

### 3. DESCRIÇÃO DOS ALGORITMOS IMPLEMENTADOS

A estrutura de busca do programa foi dividida em três abordagens clássicas:

1. **Busca em Largura (BFS):** Utiliza uma estrutura de fila FIFO (First-In, First-Out). Explora sistematicamente todos os nós na profundidade  $d$  antes de avançar para a profundidade  $d+1$ , garantindo completude e a descoberta do caminho mínimo (solução ótima).

2. **Busca em Profundidade (DFS):** Utiliza uma estrutura de pilha LIFO (Last-In, First-Out). Explora os ramos mais profundos da árvore de busca primeiro. Para mitigar o risco de caminhos redundantes e loops infinitos inerentes às idas e vindas do barco, foi adotado um conjunto de estados fechados (**closed**) para evitar a reexpansão de estados visitados.

3. **Busca com Aprofundamento Iterativo (IDS):** Opera aplicando iterativamente a busca em profundidade limitada (DLS), incrementando o limite de nível a cada rodada (0,1,2,...). Combina o baixo consumo de memória (espacial) da DFS com a otimalidade de caminho da BFS.

#### 4. ANÁLISE COMPARATIVA DOS RESULTADOS

Os algoritmos foram submetidos à execução simultânea a partir do estado inicial. Os dados obtidos foram consolidados na tabela abaixo:

| Algoritmo | Passos (Caminho) | Estados Gerados | Estados Visitados | Tempo Estimado (ms) |
|-----------|------------------|-----------------|-------------------|---------------------|
| BFS       | 11               | 29              | 14                | 0.3759              |
| DFS       | 11               | 26              | 12                | 0.0861              |
| IDS       | 11               | 152             | 163               | 0.4579              |

#### Discussão dos Resultados:

- **Tamanho do Caminho (Passos):** A **BFS** e a **IDS** alcançaram de forma previsível o menor caminho geométrico para o problema, fixado em **11 passos**. A **DFS** também finalizou em 11 passos devido à ordem estática com que os operadores foram inseridos na pilha, o que expandiu o ramo ideal logo na primeira tentativa, embora teoricamente ela não ofereça essa garantia.
- **Estados Gerados e Visitados:** A tabela evidencia o custo característico de reprocessamento da **IDS** (152 gerados e 163 visitados). Como o algoritmo reinicia a árvore do zero a cada alteração do limite de profundidade, os nós dos níveis superiores são revisitados múltiplas vezes, gerando um *overhead* computacional.
- **Tempo de Execução:** Em decorrência do tamanho reduzido do espaço de estados deste problema, todos os tempos situaram-se na escala de frações de milissegundo, apresentando execução instantânea.

## 5. Códigos e resultados:

## 1: FUNÇÃO DA BFS:

```
56
57 def breadth_first_search(problem):
58     node = Node(problem.initial_state)
59     if problem.is_goal_state(node.state): return get_path(node), 1, 1
60
61     frontier = collections.deque([node])
62     explored = set()
63     generated = 1
64
65     while frontier:
66         node = frontier.popleft()
67         if node.state in explored: continue
68
69         explored.add(node.state)
70
71         for state in problem.expand(node.state):
72             generated += 1
73             child = Node(state, node, depth=node.depth + 1)
74             if state not in explored and all(n.state != state for n in frontier):
75                 if problem.is_goal_state(state):
76                     return get_path(child), generated, len(explored) + 1
77                 frontier.append(child)
78     return None
```

| Algoritmo                                                                                                                                         | Passos | Gerados | Visitados | Tempo (ms) |
|---------------------------------------------------------------------------------------------------------------------------------------------------|--------|---------|-----------|------------|
| BFS                                                                                                                                               | 11     | 29      | 14        | 0.3759     |
| Caminho BFS: [(3, 3, 1), (3, 1, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), (0, 0, 0)] |        |         |           |            |

## 2: FUNÇÃO DA DFS

```
80 def depth_first_search(problem):
81     node = Node(problem.initial_state)
82     frontier = [node]
83     closed = set()
84     generated = 1
85
86     while frontier:
87         node = frontier.pop()
88         if node.state in closed: continue
89
90         closed.add(node.state)
91         if problem.is_goal_state(node.state):
92             return get_path(node), generated, len(closed)
93
94         for state in problem.expand(node.state):
95             generated += 1
96             child = Node(state, node, depth=node.depth + 1)
97             frontier.append(child)
98     return None
```

|                                                                                                                                                   |    |    |    |        |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|--------|
| DFS                                                                                                                                               | 11 | 26 | 12 | 0.0861 |
| Caminho DFS: [(3, 3, 1), (2, 2, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (0, 2, 1), (0, 0, 0)] |    |    |    |        |

### 3: FUNÇÕES DA IDS/DLS

```
99
100 def iterative_deepening_search(problem):
101     depth = 0
102     stats = {'gen': 1, 'vis': 0}
103     while True:
104         result = depth_limited_search(problem, depth, stats)
105         if result != 'cutoff':
106             return result, stats['gen'], stats['vis']
107         depth += 1
108
109 def depth_limited_search(problem, limit, stats):
110     node = Node(problem.initial_state)
111     return recursive_dls(node, problem, limit, stats, set())
112
113 def recursive_dls(node, problem, limit, stats, visited_in_path):
114     stats['vis'] += 1
115     if problem.is_goal_state(node.state):
116         return get_path(node)
117     if limit == 0:
118         return 'cutoff'
119
120     cutoff_occurred = False
121     visited_in_path.add(node.state)
122
123     for state in problem.expand(node.state):
124         if state not in visited_in_path:
125             stats['gen'] += 1
126             child = Node(state, node, depth=node.depth + 1)
127             result = recursive_dls(child, problem, limit - 1, stats, visited_in_path.copy())
128             if result == 'cutoff':
129                 cutoff_occurred = True
130             elif result is not None:
131                 return result
132     return 'cutoff' if cutoff_occurred else None
```

```
IDS          | 11      | 152      | 163      | 0.4579
Caminho IDS: [(3, 3, 1), (3, 1, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), (0, 0, 0)]
```

### 6. EVIDÊNCIA DA SOLUÇÃO (CAMINHO ENCONTRADO)

A sequência ordenada de transições de estados gerada para solucionar o problema a partir da busca em largura apresenta as seguintes travessias entre as margens:

1.(3, 3, 1) — Estado Inicial (3M, 3C e Barco na margem esquerda)

2.(3, 1, 0) — Dois canibais cruzam para a margem direita



- 3.(3, 2, 1) Um canibal retorna com o barco
- 4.(3, 0, 0) Dois canibais cruzam para a margem direita
- 5.(3, 1, 1) Um canibal retorna com o barco
- 6.(1, 1, 0) Dois missionários cruzam para a margem direita
- 7.(2, 2, 1) Um missionário e um canibal retornam para a esquerda
- 8.(0, 2, 0) Dois missionários cruzam para a margem direita
- 9.(0, 3, 1) Um canibal retorna com o barco
- 10.(0, 1, 0) Dois canibais cruzam para a margem direita
- 11.(1, 1, 1)\$ Um missionário e um canibal retornam
- 12.(0, 0, 0) Estado Objetivo (Todos transportados em segurança)

## 7. COMPARAÇÃO QUALITATIVA

| Algoritmo | É completo? | Por quê?                                                                                                |
|-----------|-------------|---------------------------------------------------------------------------------------------------------|
| BFS       | Sim         | Explora todas as camadas de profundidade antes de avançar, garantindo que nenhuma solução seja omitida. |

|            |            |                                                                                                                                        |
|------------|------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>DFS</b> | <b>Não</b> | <b>Corre o risco de seguir um ramo infinito ou entrar em ciclos repetitivos de ida e volta do barco sem nunca alcançar o objetivo.</b> |
| <b>IDS</b> | <b>Sim</b> | <b>Aumenta o limite de profundidade de forma gradual (1, 2, 3...), evitando que a solução seja perdida em ramos infinitos.</b>         |

**Menor Caminho:**

| <b>Algoritmo</b> | <b>Encontra o caminho mínimo?</b> | <b>Por quê?</b>                                                                                                                                  |
|------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BFS</b>       | <b>Sim</b>                        | <b>Como o custo de transição (travessia) é uniforme (=1), a primeira solução encontrada na busca em camadas é obrigatoriamente a mais curta.</b> |
| <b>DFS</b>       | <b>Não</b>                        | <b>Encontra a solução localizada mais à esquerda na árvore de busca, independentemente do tamanho do caminho ou de sua eficiência.</b>           |

|            |            |                                                                                                                                      |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>IDS</b> | <b>Sim</b> | <b>É equivalente à BFS em custos uniformes, pois só avança para um nível mais profundo após testar exaustivamente os anteriores.</b> |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------------|

## **8. INTERPRETAÇÃO GERAL**

- 1. A BFS demonstra ser a estratégia mais direta e confiável quando o objetivo principal é obter o menor caminho em um espaço de estados controlado, evitando desvios desnecessários.**
- 2. A DFS mostrou-se extremamente veloz neste teste específico devido à ordem estática dos operadores, mas provou ser uma estratégia arriscada e imprevisível, incapaz de garantir otimalidade por padrão.**
- 3. A IDS estabelece o equilíbrio ideal de engenharia: elimina o risco de caminhos infinitos (deficiência da DFS), preserva o uso linear de memória (deficiência da BFS) e assegura o caminho mínimo, pagando o preço apenas no reprocessamento redundante de nós rasos.**

**No cenário dos Missionários e Canibais, o espaço total de estados válidos é muito pequeno (apenas 16 estados operacionais possíveis). Por essa razão, todos os algoritmos encontram a resposta em frações de milissegundo, mas os fundamentos teóricos observados se alinham perfeitamente com a literatura clássica de Inteligência Artificial.**

## **9. CONCLUSÃO:**

**A resolução prática do Problema dos Missionários e Canibais permitiu validar de forma empírica as diferenças estruturais entre os algoritmos de busca sem informação. Enquanto a busca em largura destaca-se pelo compromisso com a solução ótima, a busca em profundidade foca na agressividade da descida da árvore com baixo uso de memória.**

**Por fim, o aprofundamento iterativo consolida-se como uma das soluções mais elegantes da computação clássica não informada, provando que é possível alcançar resultados ótimos e completos de forma sustentável para o hardware.**

